

# Research Notes on Actor critic algorithm

Actor critic algorithm

## >> Section 1

**Actor** The actor uses a policy function  $\pi(a|s)$ , while the critic estimates either the value function  $V(s)$ , the action-value Q-function  $Q(s,a)$ , the advantage function  $A(s,a)$ , or any combination thereof. The actor is a parameterized function  $\pi_{\theta}$ , where  $\theta$  are the parameters of the actor. The actor takes as argument the state of the environment  $s$  and produces a probability distribution  $\pi_{\theta}(\cdot|s)$ . If the action space is discrete, then  $\sum_a \pi_{\theta}(a|s) = 1$ . If the action space is continuous, then  $\int_a \pi_{\theta}(a|s) da = 1$ . The goal of policy optimization is to improve the actor. That is, to find some  $\theta$  that maximizes the expected episodic reward  $J(\theta)$ :  $J(\theta) = E_{\pi_{\theta}}[\sum_{t=0}^T \gamma^t r_t]$  where  $\gamma$  is the discount factor,  $r_t$  is the reward at step  $t$ , and  $T$  is the time-horizon (which can be infinite). The goal of policy gradient method is to optimize  $J(\theta)$  by gradient ascent on the policy gradient  $\nabla J(\theta)$ . As detailed on the policy gradient method page, there are many unbiased estimators of the policy gradient:  $\nabla_{\theta} J(\theta) = E_{\pi_{\theta}}[\sum_{0 \leq j \leq T} \nabla_{\theta} \ln \pi_{\theta}(A_j|S_j) \cdot \Psi_j | S_0 = s_0]$  where  $\Psi_j$  is a linear sum of the following:

## >> Section 2

**Variants** Asynchronous Advantage Actor-Critic (A3C): Parallel and asynchronous version of A2C. Soft Actor-Critic (SAC): Incorporates entropy maximization for improved exploration. Deep Deterministic Policy Gradient (DDPG): Specialized for continuous action spaces.

## >> Section 3

Konda, Vijay R.; Tsitsiklis, John N. (January 2003). "On Actor-Critic Algorithms". *SIAM Journal on Control and Optimization*. 42 (4): 1143–1166. doi:10.1137/S0363012901385691. ISSN 0363-0129. Sutton, Richard S.; Barto, Andrew G. (2018). *Reinforcement learning: an introduction*. Adaptive computation and machine learning series (2 ed.). Cambridge, Massachusetts: The MIT Press. ISBN

978-0-262-03924-6. Bertsekas, Dimitri P. (2019). *Reinforcement learning and optimal control* (2 ed.). Belmont, Massachusetts: Athena Scientific. ISBN 978-1-886529-39-7. Grossi, Csaba (2010). *Algorithms for Reinforcement Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning (1 ed.). Cham: Springer International Publishing. ISBN 978-3-031-00423-0. Grondman, Ivo; Busoniu, Lucian; Lopes, Gabriel A. D.; Babuska, Robert (November 2012). "A Survey of Actor-Critic Reinforcement Learning: Standard and Natural Policy Gradients". *IEEE Transactions on Systems, Man, and Cybernetics - Part C: Applications and Reviews*. 42 (6): 1291–1307. Bibcode:2012ITHMS..42.1291G. doi:10.1109/TSMCC.2012.2218595. ISSN 1094-6977.

## >> Section 4

$\gamma^j (\sum_{k=0}^{n-1} \gamma^k R_{j+k} + \gamma^n V_{\pi_{\theta}}(S_{j+n}) - V_{\pi_{\theta}}(S_j))$  : TD(n) learning.

## >> Section 5

$\gamma^j (R_j + \gamma R_{j+1} + \gamma^2 V_{\pi_{\theta}}(S_{j+2}) - V_{\pi_{\theta}}(S_j))$  : TD(2) learning.

## >> Section 6

The actor-critic algorithm (AC) is a family of reinforcement learning (RL) algorithms that combine policy-based RL algorithms such as policy gradient methods, and value-based RL algorithms such as value iteration, Q-learning, SARSA, and TD learning. An AC algorithm consists of two main components: an "actor" that determines which actions to take according to a policy function, and a "critic" that evaluates those actions according to a value function. Some AC algorithms are on-policy, some are off-policy. Some apply to either continuous or discrete action spaces. Some work in both cases.

## >> Section 7

$\gamma^j (R_j + \gamma V_{\pi_{\theta}}(S_{j+1}) - V_{\pi_{\theta}}(S_j))$  : TD(1) learning.

## >> Section 8

$\gamma^j \sum_{n=1}^{\infty} \lambda^{n-1} (1 - \lambda) (\sum_{k=0}^{n-1} \gamma^k R_{j+k} + \gamma^n V_{\pi_{\theta}}(S_{j+n}) - V_{\pi_{\theta}}(S_j))$  : TD( $\lambda$ ) learning, also known as GAE (generalized

# Research Notes on Actor critic algorithm

## Actor critic algorithm

advantage estimate). This is obtained by an exponentially decaying sum of the TD(n) learning terms.

### >> Section 9

$\gamma \sum_{j \leq i \leq T} (\gamma^{i-j} R_i) - b(S_j)$  : the REINFORCE with baseline algorithm. Here  $b$  is an arbitrary function.

### >> Section 10

Critic In the unbiased estimators given above, certain functions such as  $V_{\pi_{\theta}}$ ,  $Q_{\pi_{\theta}}$ ,  $A_{\pi_{\theta}}$  appear. These are approximated by the critic. Since these functions all depend on the actor, the critic must learn alongside the actor. The critic is learned by value-based RL algorithms. For example, if the critic is estimating the state-value function  $V_{\pi_{\theta}}(s)$ , then it can be learned by any value function approximation method. Let the critic be a function approximator  $V_{\phi}(s)$  with parameters  $\phi$ . The simplest example is TD(1) learning, which trains the critic to minimize the TD(1) error:  $\delta_i = R_i + \gamma V_{\phi}(S_{i+1}) - V_{\phi}(S_i)$ . The critic parameters are updated by gradient descent on the squared TD error:  $\phi \leftarrow \phi - \alpha \nabla_{\phi} (\delta_i)^2 = \phi + \alpha \delta_i \nabla_{\phi} V_{\phi}(S_i)$  where  $\alpha$  is the learning rate. Note that the gradient is taken with respect to the  $\phi$  in  $V_{\phi}(S_i)$  only, since the  $\phi$  in  $\gamma V_{\phi}(S_{i+1})$  constitutes a moving target, and the gradient is not taken with respect to that. This is a common source of error in implementations that use automatic differentiation, and requires "stopping the gradient" at that point. Similarly, if the critic is estimating the action-value function  $Q_{\pi_{\theta}}$ , then it can be learned by Q-learning or SARSA. In SARSA, the critic maintains an estimate of the Q-function, parameterized by  $\phi$ , denoted as  $Q_{\phi}(s, a)$ . The temporal difference error is then calculated as  $\delta_i = R_i + \gamma Q_{\phi}(S_{i+1}, A_{i+1}) - Q_{\phi}(S_i, A_i)$ . The critic is then updated by  $\theta \leftarrow \theta + \alpha \delta_i \nabla_{\theta} Q_{\theta}(S_i, A_i)$ . The advantage critic can be trained by training both a Q-function  $Q_{\phi}(s, a)$  and a state-value function  $V_{\phi}(s)$ , then let  $A_{\phi}(s, a) = Q_{\phi}(s, a) - V_{\phi}(s)$ .

$A_{\phi}(s, a) = Q_{\phi}(s, a) - V_{\phi}(s)$ . Although, it is more common to train just a state-value function  $V_{\phi}(s)$ , then estimate the advantage by  $A_{\phi}(S_i, A_i) \approx \sum_{j=0}^{n-1} \gamma^j R_{i+j} + \gamma^n V_{\phi}(S_{i+n}) - V_{\phi}(S_i)$ . Here,  $n$  is a positive integer. The higher  $n$  is, the more lower is the bias in the advantage estimation, but at the price of higher variance. The Generalized Advantage Estimation (GAE) introduces a hyperparameter  $\lambda$  that smoothly interpolates between Monte Carlo returns ( $\lambda = 1$ , high variance, no bias) and 1-step TD learning ( $\lambda = 0$ , low variance, high bias). This hyperparameter can be adjusted to pick the optimal bias-variance trade-off in advantage estimation. It uses an exponentially decaying average of n-step returns with  $\lambda$  being the decay strength.